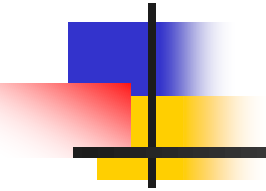
Fundamental of Software Engineering (SENG 1061)



Chapter 05

Software Design

Topics Covered

- Software Design
- Software Design Levels
- Software Design Techniques
- Software Design Strategies
- Software Design Approaches
- Software Architectures

Objectives

- At the end of this session student will be able to:
  - Understand what does mean software design
  - Identify software design techniques , strategies and approaches
  - Apply effective software design methodology to design a software system
  - Design specific and measurable software design to ensure software requirements.

Software Design

- A process to transform user requirements into some suitable form, which helps the programmer in software coding and implementation.
  - For assessing user requirements, a Software Requirement Specification (SRS) document is created
  - whereas for coding and implementation, there is a need of more specific and detailed requirements in software terms.
- The output of this process can directly be used into implementation in programming languages.
- Software design is the first step in Software Design Life Cycle (SDLC), which moves the concentration from problem domain to solution domain.
- It tries to specify how to fulfill the requirements mentioned in SRS.

Software Design Levels

- **Architectural Design**
  - defines the relationships among the major structural elements of the software, it is derived from the class-based and flow-oriented elements of the analysis model
  - It identifies the software as a system with many components interacting with each other.
  - At this level, the designers get the idea of proposed solution domain.
- **High-level Design**
  - breaks the 'single entity-multiple component' concept of architectural design into less-abstracted view of sub-systems and modules and depicts their interaction with each other.
  - focuses on how the system along with all of its components can be implemented in forms of modules.
  - It recognizes modular structure of each sub-system and their relation and interaction among each other.
- **Detailed Design**
  - deals with the implementation part of what is seen as a system and its sub-systems in the previous two designs.
  - It is more detailed towards modules and their implementations.
  - It defines logical structure of each module and their interfaces to communicate with other modules.

Why design is so important?

- It is place where quality is fostered.
- It provides us with representation of software that can be assessed for quality.
- Only way that can accurately translate a customer's requirements into a finished software product.
- It serves as foundation for all software engineering activities.
- Without design difficult to assess:
  - Risk
  - Test
  - Quality

Design Process and Design Quality

- S/w design is an iterative process through which requirements are translated into a “blueprint” for constructing the s/w.
- As design iteration occur, subsequent refinement leads to design representation at much lower levels of abstraction.

Goal of design process

- The design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer.
- The design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.
- The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

8

Design quality attributes

- Acronym FURPS –
  - Functionality
  - Usability
  - Reliability
  - Performance
  - Supportability

Cont.…

- Functionality – is assessed by evaluating the feature set and capabilities of the program.
  - Functions that are delivered and security of the overall system.
- Usability – assessed by considering human factors, consistency & documentation.
- Reliability – evaluated by
  - measuring the frequency and severity of failure.
  - Accuracy of output results.
  - Ability to recover from failure and predictability of the program.
- Performance -measured by processing speed, response time, resource, efficiency.
- Supportability – combines the ability to extend the program (extensibility), adaptability and serviceability.

Conti…

- A design should exhibit an architecture that
  - as been created using recognizable architectural styles or patterns,
  - is composed of components that exhibit good design characteristics and
  - can be implemented in an evolutionary fashion
- A design should be modular; that is, the software should be logically partitioned into elements or subsystems
- A design should contain distinct representations of data, architecture, interfaces, and components.
- A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.
- A design should lead to components that exhibit independent functional characteristics

Conti…

- The design process should not suffer from ‘tunnel vision.’

  Designer should consider alternative approaches.

- The design should be traceable to the analysis model - a single element of the design model often traces to multiple requirements, it is necessary to have a means for tracking how requirements have been satisfied by the design model.

- The design should “minimize the intellectual distance” between the software and the problem as it exists in the real world. – design should be self-explanatory

Software Design Techniques

- **Modularization**
  - A technique to divide a software system into multiple discrete and independent modules, which are expected to be capable of carrying out task(s) independently.
  - These modules may work as basic constructs for the entire software.
  - Designers tend to design modules such that they can be executed and/or compiled separately and independently.
- Modular design unintentionally follows the rules of **'divide and conquer' problem-solving strategy** this is because there are many other benefits attached with the modular design of a software.

Advantages

- Smaller components are easier to maintain
- Program can be divided based on functional aspects
- Desired level of abstraction can be brought in the program
- Components with high cohesion can be re-used again
- Concurrent execution can be made possible

Concurrency

- In software design, concurrency is implemented by splitting the software into multiple independent units of execution, like modules and executing them in parallel.
  - In other words, concurrency provides capability to the software to execute more than one part of code in parallel to each other.
- It is necessary for the programmers and designers to recognize those modules, which can be made parallel execution.
  - Example
    - The **spell check feature in word processor** is a module of software, which runs along side the word processor itself.

Coupling and Cohesion

- When a software program is modularized, its tasks are divided into several modules based on some characteristics.
  - **modules** are set of instructions put together in order to achieve some tasks.
    - They are though, considered as single entity but may refer to each other to work together.
    - There are measures by which the quality of a design of modules and their interaction among them can be measured.
      - These measures are called coupling and cohesion.

Coupling and Cohesion

- **Cohesion**
  - A measure that defines the degree of intra-dependability within elements of a module.
  - Grouping of all functionally related elements.
  - The greater the cohesion, the better is the program design.
- **Coupling**
  - A measure that defines the level of inter-dependability among modules of a program.
  - It tells at what level the modules interfere and interact with each other.
  - The lower the coupling, the better the program.

Software Design Strategies/ Methodology

- Software design is a process to conceptualize the software requirements into software implementation.
- Software design takes the user requirements as challenges and tries to find optimum solution.
- While the software is being conceptualized, a plan is chalked out to find the best possible design for implementing the intended solution.
- There are multiple variants of software design.

Structured Design

- It is a conceptualization of problem into several well-organized elements of solution.
- It is basically concerned with the solution design.
- Benefit of structured design is, it gives better understanding of how the problem is being solved.
- also makes it simpler for designer to concentrate on the problem more accurately.
- is mostly based on ‘divide and conquer’ strategy where a problem is broken into several small problems and each small problem is individually solved until the whole problem is solved.

Structured Design

- The small pieces of problem are solved by means of solution modules.
- emphasis that these modules be well organized in order to achieve precise solution.
- These modules are arranged in hierarchy.
- They communicate with each other.
  - A good structured design always follows some rules for communication among multiple modules, namely -
  - **Cohesion**
  - **Coupling**
- A good structured design has high cohesion and low coupling arrangements.

Function Oriented Design

- The system is comprised of many smaller sub-systems known as functions.
  - These functions are capable of performing significant task in the system.
  - The system is considered as top view of all functions.
- **Characteristic of Function oriented design**
  - inherits some properties of structured design where divide and conquer methodology is used.
  - This design mechanism divides the whole system into smaller functions, which provides means of abstraction by concealing the information and their operation.
  - These functional modules can share information among themselves by means of information passing and using information available globally.
  - when a program calls a function, the function changes the state of the program, which sometimes is not acceptable by other modules.
    - Function oriented design works well where the system state does not matter and program/functions work on input rather than on a state.



Conti…

- **Design Process**
  - The whole system is seen as how data flows in the system by means of data flow diagram.
  - DFD depicts how functions changes data and state of entire system.
  - The entire system is logically broken down into smaller units known as functions on the basis of their operation in the system.
  - Each function is then described at large.

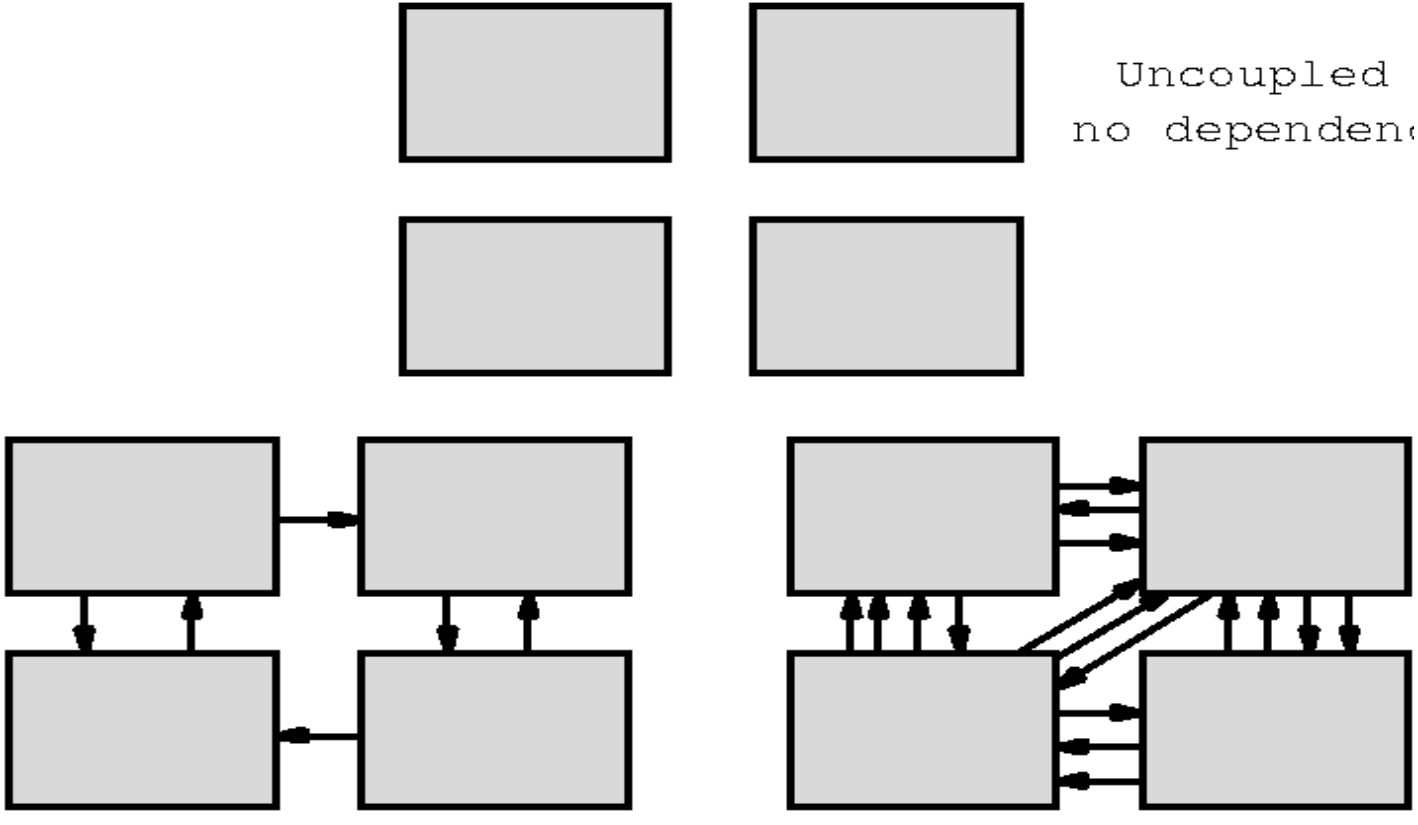
Conti…

- Data-flow design
  - Model the data processing in the system using data-flow diagrams
- Structural decomposition
  - Model how functions are decomposed to sub-functions using graphical structure charts
- Detailed design
  - The entities in the design and their interfaces are described in detail. These may be recorded in a data dictionary and the design expressed using a PDL

Structural decomposition

- Structural decomposition is concerned with developing a model of the design which shows the dynamic structure i.e. function calls
- The aim of the designer should be to derive design units which are highly cohesive and loosely coupled
- In essence, a data flow diagram is converted to a structure chart

Coupling and cohesion



Object Oriented Design

- Object oriented design works around the entities and their characteristics instead of functions involved in the software system.
- This design strategies focuses on entities and its characteristics.
- The whole concept of software solution revolves around the engaged entities.

Object Oriented Design: Concepts

- **Objects**
  - All entities involved in the solution design are known as objects.
    - For example, person, banks, company and customers are treated as objects.
    - Every entity has some attributes associated to it and has some methods to perform on the attributes.
- **Classes**
  - A class is a generalized description of an object. An object is an instance of a class.
  - Class defines all the attributes, which an object can have and methods, which defines the functionality of the object.
- In the solution design, attributes are stored as variables and functionalities are defined by means of methods or procedures.

Conti…

- **Encapsulation**
  - In OOD, the attributes (data variables) and methods (operation on the data) are bundled together is called encapsulation.
  - Encapsulation not only bundles important information of an object together, but also restricts access of the data and methods from the outside world.
    - This is called information hiding.
- **Inheritance**
  - OOD allows similar classes to stack up in hierarchical manner where the lower or sub-classes can import, implement and re-use allowed variables and methods from their immediate super classes. This property of OOD is known as inheritance.
  - This makes it easier to define specific class and to create generalized classes from specific ones.

Conti…

- **Polymorphism**
  - OOD languages provide a mechanism where methods performing similar tasks but vary in arguments, can be assigned same name.
  - This is called polymorphism, which allows a single interface performing tasks for different types.
  - Depending upon how the function is invoked, respective portion of the code gets executed.

Conti…

- **Design Process**
  - Software design process can be perceived as series of well-defined steps. Though it varies according to design approach (function oriented or object oriented, yet It may have the following steps involved:
  - A solution design is created from requirement or previous used system and/or system sequence diagram.
  - Objects are identified and grouped into classes on behalf of similarity in attribute characteristics.
  - Class hierarchy and relation among them is defined.
  - Application framework is defined.

Object Oriented Design
System design activities: From objects to subsystems

1. Identify design goals from the nonfunctional requirements
2. Design an initial subsystem decomposition
3. Map subsystems to processors and components
4. Decide storage
5. Define access control policies
6. Select a control flow mechanism
7. Identify boundary conditions

1. Identifying design goals

- Is the **first step** of system design, It **identifies the qualities** that our system should focus on; which are mostly inferred from the **nonfunctional requirements** or from **the application domain.**
- **Example: considering the** the nonfunctional requirements for MyTrip

Design goals for MyTrip

- **Reliability:** MyTrip should be reliable [generalization of nonfunctional requirement 2].
- **Fault Tolerance:** MyTrip should be fault tolerant to loss of connectivity with the routing service [rephrased nonfunctional requirement 2].
- **Security:** MyTrip should be secure, i.e., not allow other drivers or nonauthorized users to access another driver's trips [deduced from application domain].
- **Modifiability:** MyTrip should be modifiable to use different routing services [**anticipation of change by developers**].

Cont…

- list a number of possible design criteria.
  - ***Performance***
  - ***dependability***
  - ***cost***
  - ***maintenance***
  - ***end user criteria.***

1. **Performance criteria:** include the speed and space requirements imposed on the system.

Design criterion	Definition
Response time	How soon is a user request acknowledged after the request has been issued?
Throughput	How many tasks can the system accomplish in a fixed period of time?
Memory	How much space is required for the system to run?

Cont..

2. **Dependability criteria:** determine how much effort should be expended in **minimizing system crashes** and their consequences. How often can the system crash? How **available** to the user should the system be? Are there **safety issues** associated with system crashes? Are there **security risks** associated with the system environment?

Design criterion	Definition
Robustness	Ability to survive invalid user input
Reliability	Difference between specified and observed behavior.
Availability	Percentage of time system can be used to accomplish normal tasks.
Fault tolerance	Ability to operate under erroneous conditions.
Security	Ability to withstand malicious attacks
Safety	Ability to not endanger human lives, even in the presence of errors and failures.

Cont…

3. **Cost criteria:** include the cost to develop the system, to deploy it, and to administer it.

Design criterion	Definition
Development cost	Cost of developing the initial system
Deployment cost	Cost of installing install the system and training the users.
Upgrade cost	Cost of translating data from the previous system. This criteria results in backward compatibility requirements.
Maintenance cost	Cost required for bug fixes and enhancements to the system
Administration cost	Money required to administer the system.

Cont…

4. **Maintenance criteria:** determine how difficult it is to change the system after deployment. How easily can **new functionality** be added? How easily can existing functions be **revised**? Can the system be **adapted to a different application domain**? How much effort will be required to port the system to a different platform?

Design criterion	Definition
Extensibility	How easy is it to add the functionality or new classes of the system?
Modifiability	How easy is it to change the functionality of the system?
Adaptability	How easy is it to port the system to different application domains?
Portability	How easy is it to port the system to different platforms?
Readability	How easy is it to understand the system from reading the code?
Traceability of requirements	How easy is it to map the code to specific requirements?

Cont…

5. **End user criteria:** include qualities that are desirable from a users' point of view that have not yet been covered under the **performance** and **dependability** criteria.

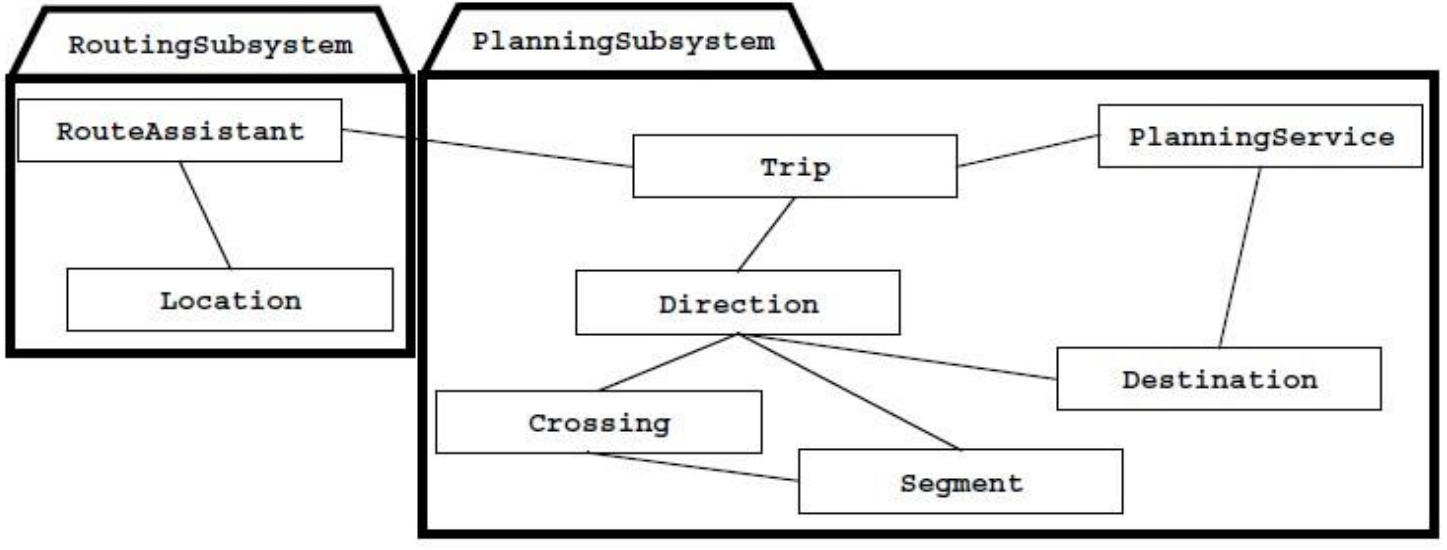
Design criterion	Definition
Utility	How well does the system support the work of the user?
Usability	How easy is it for the user to use the system?

- **N.B.** Developers need to prioritize design goals and trade them off against each other. E.g it is unrealistic to develop software that is **safe, secure, and cheap. Some of the common trade-off are:**
  - Space vs. speed
  - Delivery time vs. functionality
  - Delivery time vs. quality
  - Delivery time vs. staffing

2. Identifying subsystems

- Finding subsystems during system design has many similarities to finding objects during analysis: *It is a volatile activity driven by heuristics.*
- The initial subsystem decomposition should be derived from the functional requirements.
- *For example* :in the MyTrip system, we identify two major groups of objects:
  1. PlanTrip use cases
  2. ExecuteTrip use case.
  - *N.B.* The Trip, Direction, Crossing, Segment, and Destination classes are shared between both use cases.

Cont ...



PlanningSubsystem The `PlanningSubsystem` is responsible for constructing a `Trip` connecting a sequence of `Destinations`. The `PlanningSubsystem` is also responsible for responding to replan requests from `RoutingSubsystems`.

RoutingSubsystem The `RoutingSubsystem` is responsible for downloading a `Trip` from the `PlanningService` and executing it by giving `Directions` to the driver based on its `Location`.

Cont…

- Another heuristic for subsystem identification is to keep functionally related objects together.

Heuristics for grouping objects into subsystems

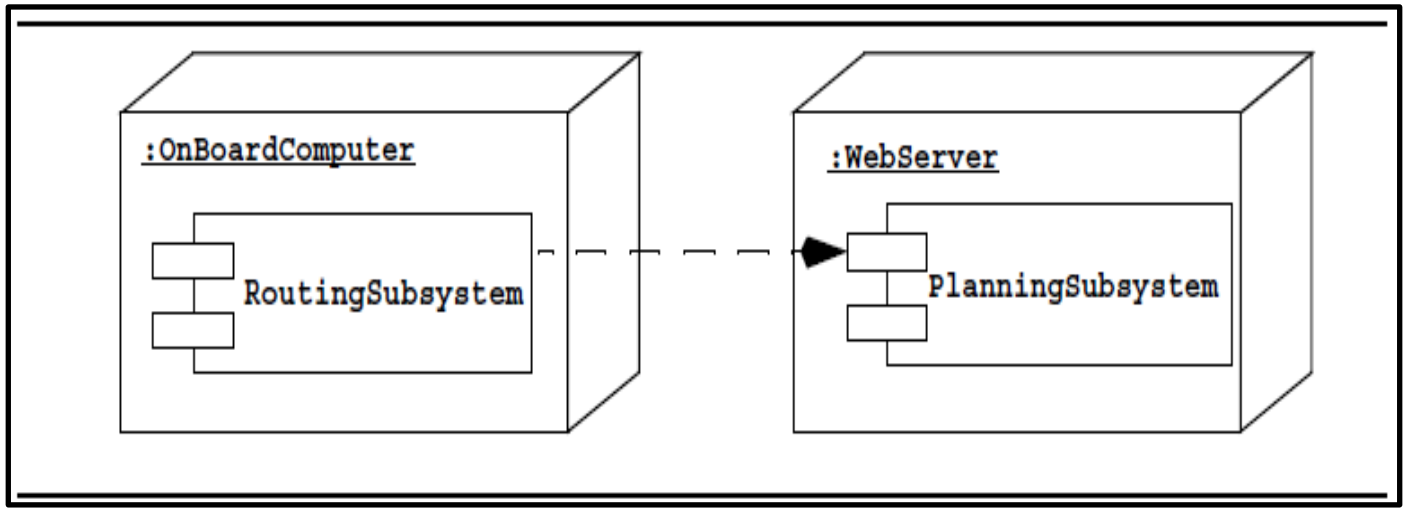
- Assign objects identified in one use case into the same subsystem
- Create a dedicated subsystem for objects used for moving data among subsystems
- Minimize the number of associations crossing subsystem boundaries
- All objects in the same subsystem should be functionally related

3. Mapping subsystems to processors and components

- ***Selecting a hardware configuration and a platform***
  - Many web based systems need multiple computers and interconnect multiple distributed users.
  - These computers are modeled as **nodes** in UML **deployment diagrams.**
  - Select a hardware configuration also includes selecting a **virtual machine** onto which the system should be built. It includes OS, s/w (DBMS, communication packages).
- **Exaple** In MyTrip, we deduce from the requirements that **PlanningSubsystem** and **RoutingSubsystem** run on two different nodes:

Cont…

- In the **MyTrip** subsystem We select a **Unix machine** as the virtual machine for the :**WebServer** and the **Web browsers** Netscape and Internet Explorer as the virtual machines for the :**OnBoardComputer.**



- **Fig.** Allocation of MyTrip subsystems to hardware (UML deployment diagram). RoutingSubsystem runs on the OnBoardComputer while PlanningSubsystem runs on a WebServer.

4. Defining persistent data stores

- Where and how data is stored in the system impacts the system decomposition.
- The selection of a specific database management system can also have implications on the overall control strategy and concurrency management.
- we first need to identify which objects need to be persistent. The persistency of objects is directly inferred from the application domain.
  - E.g. In Mytrip - current trip in Rroutngsubsystem-file, location X

    - entire trip in PlanningSubsystem-database

5. Defining access control

- In multiuser systems, different actors have access to different functionality and data. We modeled these distinctions by associating different **use cases** to different actors.

- During system design, we model access by examining the object model, by **determining which objects are shared among actors**, and by defining how actors can control access.
- Depending on the **security requirements** on the system, we also define **how actors are authenticated** to the system (i.e., how actors prove to the system who they are) and how selected data in the system should be **encrypted**.

Cont…

CommunicationSubsystem	The `CommunicationSubsystem` is responsible for transporting `Trips` from the `PlanningSubsystem` to the `RoutingSubsystem`. *The CommunicationSubsystem uses the Driver associated with the Trip being transported for selecting a key and encrypting the communication traffic.*
PlanningSubsystem	The `PlanningSubsystem` is responsible for constructing a `Trip` connecting a sequence of `Destinations`. The `PlanningSubsystem` is also responsible for responding to replan requests from `RoutingSubsystem`. *Prior to processing any requests, the PlanningSubsystem authenticates the Driver from the RoutingSubsystem. The authenticated Driver is used to determine which Trips can be sent to the corresponding RoutingSubsystem.*
Driver	*A Driver represents an authenticated user. It is used by the CommunicationSubsystem to remember keys associated with a user and by the PlanningSubsystem to associate Trips with users.*

- Fig. Revisions to the design model stemming from the decision to authenticate Drivers and encrypt communication traffic. The text added to the model is in *italics*.

Cont…

- **Access matrix:** Is a table where the *rows* of the matrix represents the **actors** of the system. The *columns* represent **classes** whose access we control.
- An entry (class, actor) in the access matrix is called an **access right** and lists the **operations** that can be executed on instances of the class or by the actor.

Objects Actors	Corporation	LocalBranch	Account

`Teller`		`lookupLocalAccount()`	`postSmallDebit()`
			`postSmallCredit()`
			`lookupBalance()`

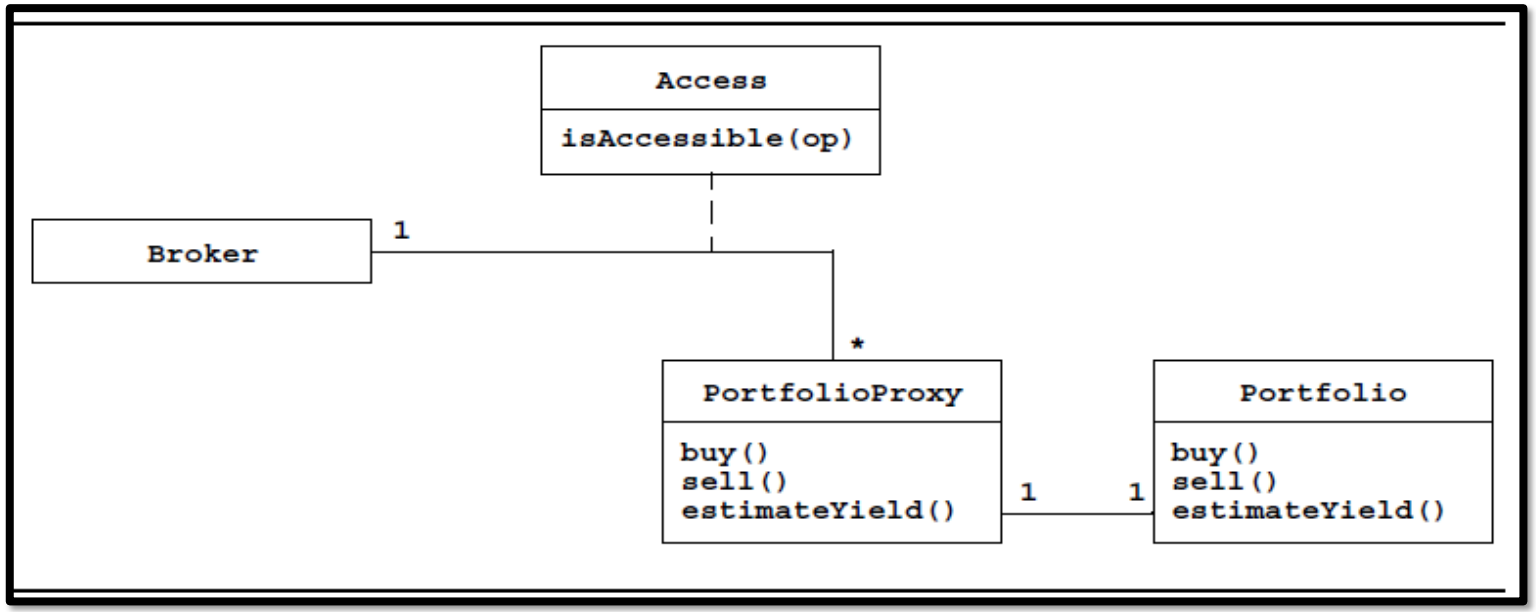
`Manager`		`lookupLocalAccount()`	`postSmallDebit()`
			`postSmallCredit()`
			`postLargeDebit()`
			`postLargeCredit()`
			`examineBalance()`
			`examineHistory()`

`Analyst`	`examineGlobalDebits()` `examineGlobalCredits()`	`examineLocalDebits()`	

- Fig. Access matrix for a banking system.

Cont…

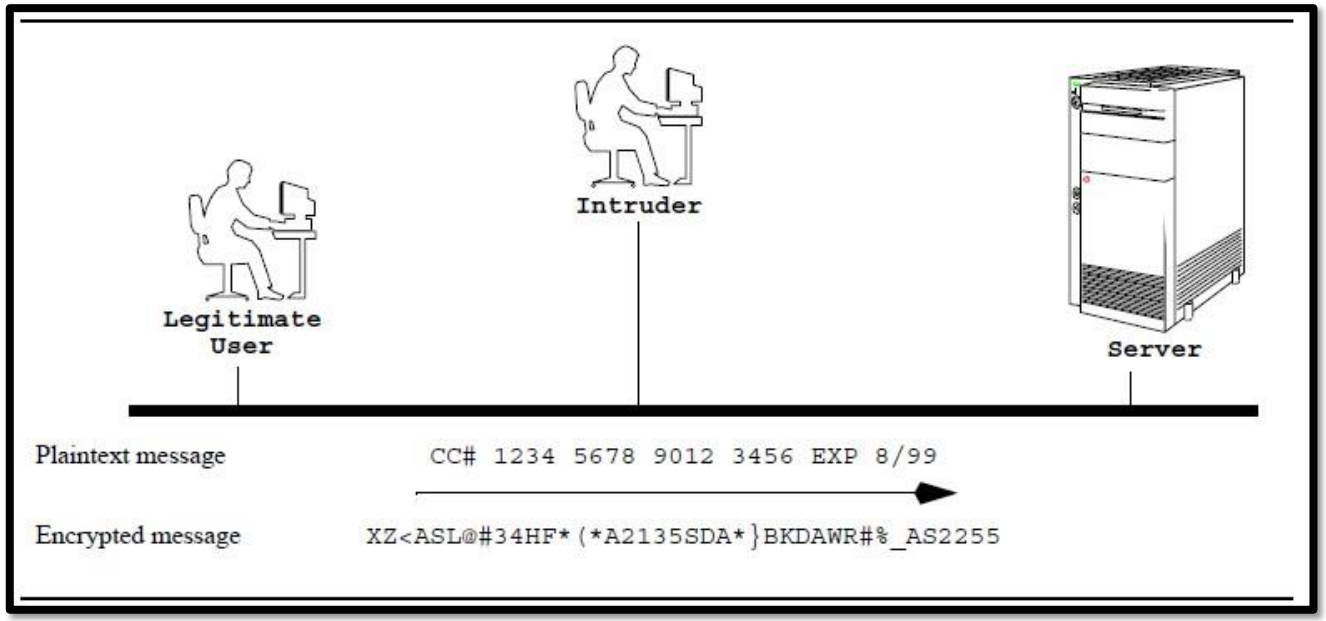
- An access matrix only represents **static access control.** This means that access rights can be modeled as **attributes** of the objects of the system.
- In a model if the access rights is allocated dynamically in the system, this type of access is called **dynamic access control.**



- **Fig.** Dynamic access implemented with a protection Proxy.

Cont…

- In an environment where resources are shared among **multiple users, authentication is usually not sufficient**.
- **Encryption** is used to prevent such unauthorized accesses. Using an encryption algorithm, we can translate a message, called **plaintext,** into a encrypted message, called a **ciphertext,**



6. Designing the global control flow

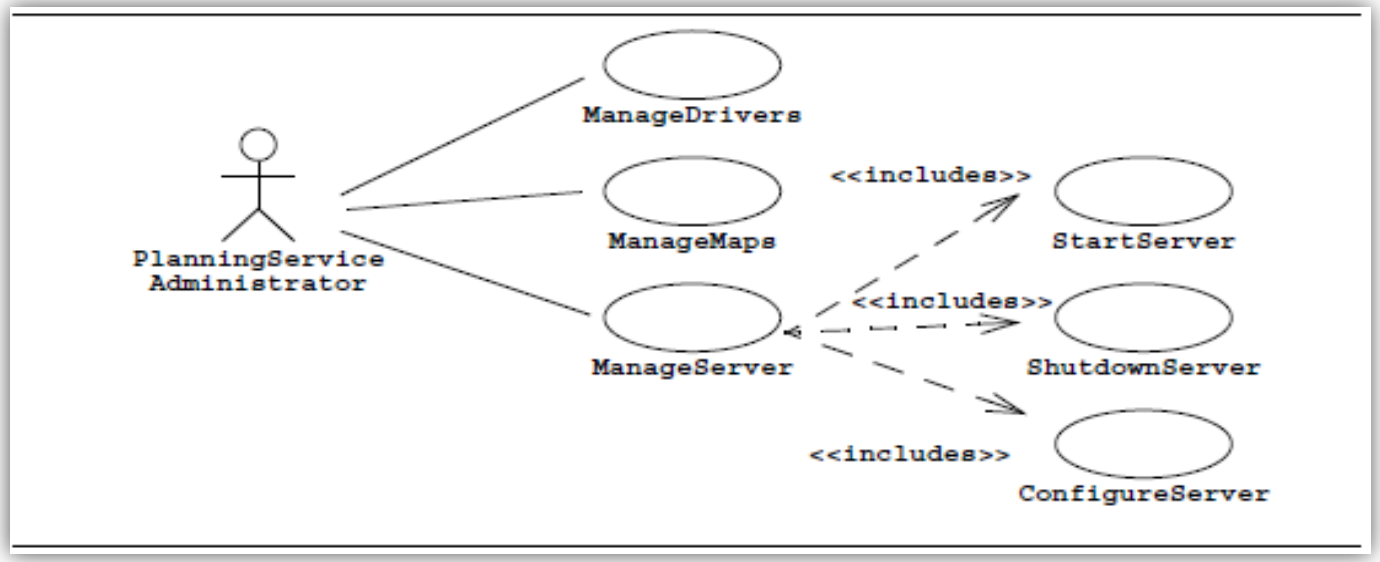
- **Control flow :** is the sequencing of **actions in a system**.
- In an object-oriented systems, sequencing actions includes deciding **which operations** should be executed and **in which order.**
  - These decisions are based on **external events** generated by an **actor** or on the passage **of time**.
- Global control describes how the global software control is implemented. In particular, this section should describe how requests are initiated and how subsystems synchronize.

7.Identifying boundary conditions

- It is deciding how the system is **started, initialized, and shut down**, and how to deal with **major failures**, such as data corruption, whether they are caused by a **software error** or a **power outage**.
- Considering the MyTrip system our system design doden't address the following questions:
  - How MyTrip is initialized and How are maps loaded into the PlanningService?
  - How is MyTrip installed in the car?
  - How does MyTrip know which PlanningService to connect to?
  - How are drivers added to the Planning Service?
- We discover a set of **use cases** that has not been specified. We call these the **system** administration use cases, which specify the system during startup and shutdown phases.

Cont…

- Modify the analysis model for MyTrip to include the administration use cases : **ManageDrivers,** to add, remove, and edit drivers; **ManageMaps**, to add, remove, and update maps used to generate trips; and **ManageServer,** to perform routine configuration, start-up, and shutdown. StartServer, part of ManageServer.



- Fig. Administration use cases for MyTrip (UML use case diagram).

Software Design Approaches

- **Top Down Design**
  - Top-down design takes the whole software system as one entity and then decomposes it to achieve more than one sub-system or component based on some characteristics.
    - Each sub-system or component is then treated as a system and decomposed further.
    - This process keeps on running until the lowest level of system in the top-down hierarchy is achieved.
  - Top-down design starts with a generalized model of system and keeps on defining the more specific part of it. When all components are composed the whole system comes into existence.
  - Top-down design is more suitable when the software solution needs to be designed from scratch and specific details are unknown.

Software Design Approaches

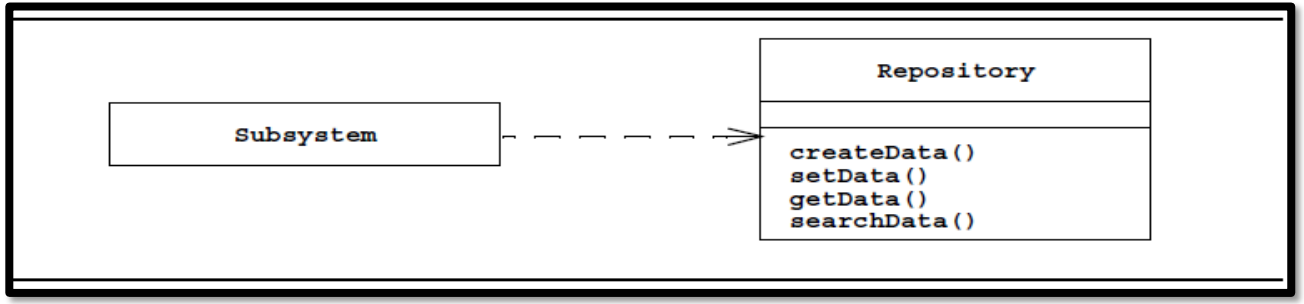
- **Bottom-up Design**
  - The bottom up design model starts with most specific and basic components. It proceeds with composing higher level of components by using basic or lower level components. It keeps creating higher level components until the desired system is not evolved as one single component. With each higher level, the amount of abstraction is increased.
  - Bottom-up strategy is more suitable when a system needs to be created from some existing system, where the basic primitives can be used in the newer system.
- Both, top-down and bottom-up approaches are not practical individually.
  - Instead, a good combination of both is used.

Software architecture

- A **software architecture** includes the system decomposition, the global control flow, error-handling policies and inter subsystem communication protocols [Shaw & Garlan, 1996].

Some of architectural type

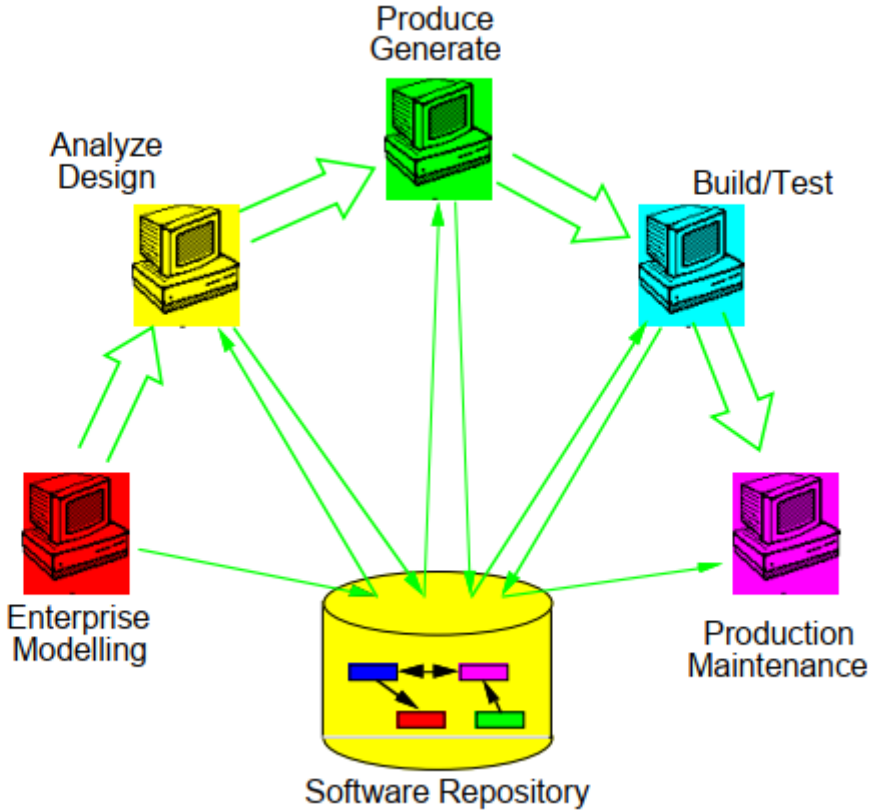
1. ***Repository architecture:*** An architecture where subsystems access and modify data from a **single data structure** called the central **repository.**
   - Control flow can be dictated either by the central repository
   - The repository architecture is typical for **database management systems**, such as a payroll system or a bank system.



- Fig. Repository architecture (UML class diagram). Every subsystem depends only on a central data structure called the repository. The repository in turn, has no knowledge of

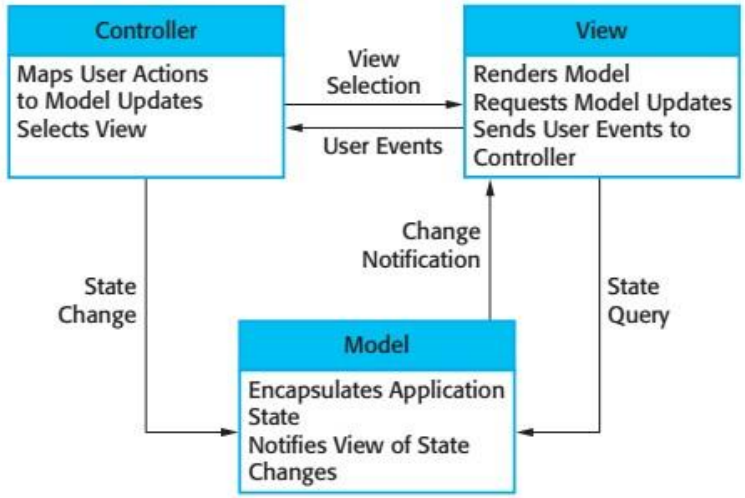
Conti…

Repositories are very important for data integration, are being introduced in a variety of applications, including software development, CAD etc.



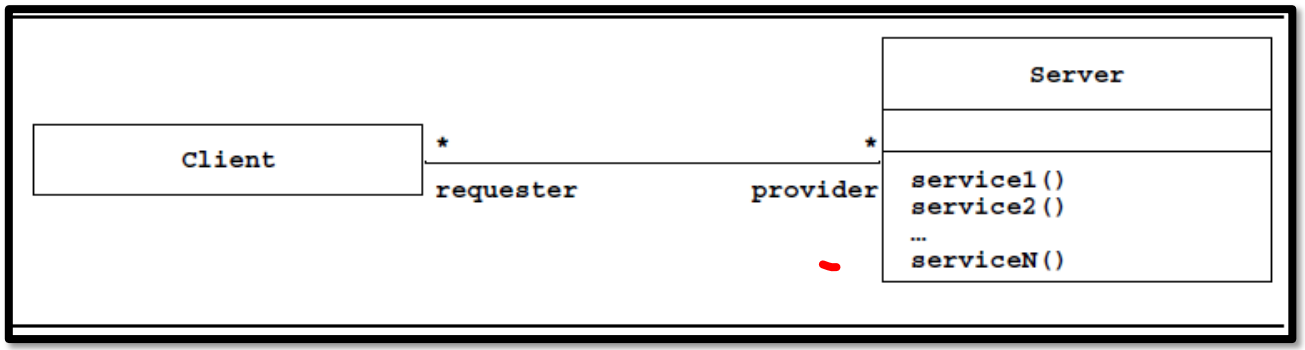
2. Model/View/Controller:

- In the Model/View/Controller (MVC) architecture subsystems are classified into three different types:
  1. **model subsystems** are responsible for maintaining domain knowledge,
     - It is developed such that they do not depend on any view or controller subsystem.
     - It maintain the central data structure.
  2. **view subsystems** are responsible for displaying it to the user, and
  3. **Controller subsystems** are responsible for managing the sequence of interactions with the user. It gather input from the user and send message to the model.



3. Client/server architecture

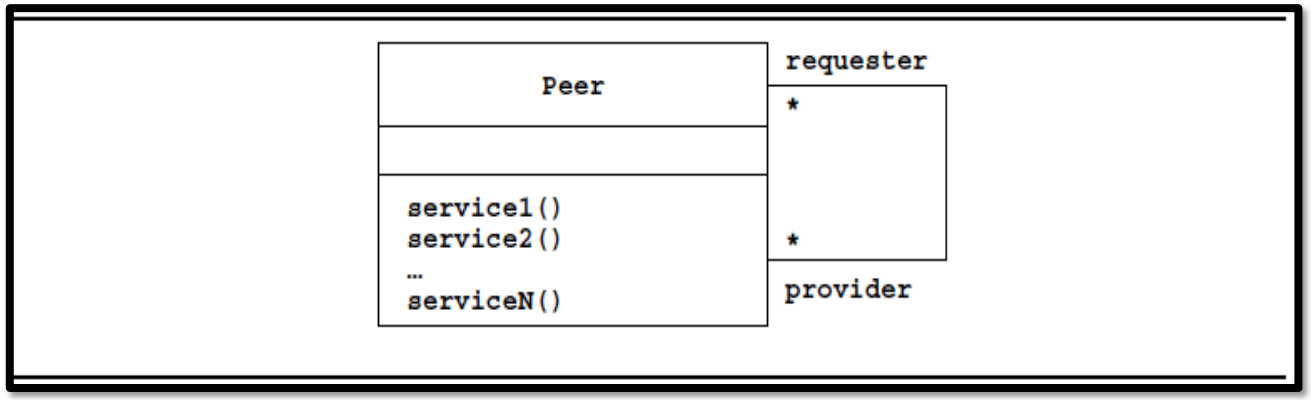
- In the client/server architecture a subsystem, the **server,** provides services to instances of other subsystems called the **clients,** which are responsible for interacting with the user.
- The request for a service is usually done via a remote procedure call mechanism or a common object broker.
- The client/server architecture is a generalization of the repository architecture.



- Fig. Client/server architecture (UML class diagram).

4. A peer-to-peer architecture

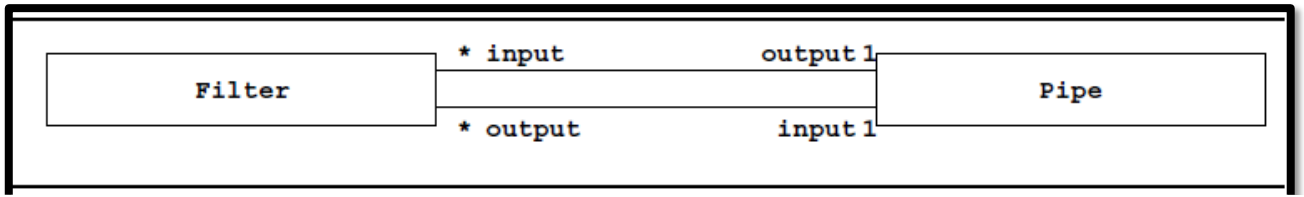
- It is a generalization of the client/server architecture in which subsystems can act both as client or as servers, in the sense that each subsystem can request and provide services.
- The control flow within each subsystem is independent from the others except for synchronizations on requests.



- Fig. Peer-to-peer architecture (UML class diagram).

5. Pipe and filter architecture

- In this architecture subsystems process data received from a set of inputs and send results to other subsystems via a set of outputs.
- The subsystems are called **filters,** and the associations between the subsystems are called **pipes.**
- Each filter only knows the content and the format of the data received on the input pipes, not the filters that produced them.
- Each filter is executed concurrently and synchronization is done via the pipes.
- Most operating systems and programming languages provide a data stream mechanism; .
- A Filter can have many inputs and outputs. A Pipe connects one of the outputs of a Filter to one of the inputs of another Filter.



Documenting system design

System Design Document

1. Introduction
   - 1.1 Purpose of the system
   - 1.2 Design goals
   - 1.3 Definitions, acronyms, and abbreviations
   - 1.4 References
   - 1.5 Overview
2. Current software architecture
3. Proposed software architecture
   - 3.1 Overview
   - 3.2 Subsystem decomposition
   - 3.3 Hardware/software mapping
   - 3.4 Persistent data management
   - 3.5 Access control and security
   - 3.6 Global software control
   - 3.7 Boundary conditions
4. Subsystem services

Glossary

Thank You!

Q?